Red Hat OpenShift Data Foundation 4.21

Deploying OpenShift Data Foundation using Google Cloud

Instructions on deploying OpenShift Data Foundation on existing Red Hat OpenShift Container Platform Google Cloud clusters

Last Updated: 2026-07-08

Red Hat OpenShift Data Foundation 4.21 Deploying OpenShift Data Foundation using Google Cloud

Instructions on deploying OpenShift Data Foundation on existing Red Hat OpenShift Container Platform Google Cloud clusters

Legal Notice

Abstract

Read this document for instructions about how to install Red Hat OpenShift Data Foundation using Red Hat OpenShift Container Platform on Google Cloud.

Table of Contents

PROVIDING FEEDBACK ON RED HAT DOCUMENTATION

We appreciate your input on our documentation. Do let us know how we can make it better.

To give feedback, create a Jira ticket:

1. Log in to the Jira.
2. Click **Create** in the top navigation bar
3. Enter a descriptive title in the Summary field.
4. Enter your suggestion for improvement in the Description field. Include links to the relevant parts of the documentation.
5. Select **Documentation** in the Components field.
6. Click Create at the bottom of the dialogue.

PREFACE

Red Hat OpenShift Data Foundation supports deployment on existing Red Hat OpenShift Container Platform (RHOCP) Google Cloud clusters.

NOTE

Only internal OpenShift Data Foundation clusters are supported on Google Cloud. See Planning your deployment for more information about deployment requirements.

To deploy OpenShift Data Foundation in internal mode, start with the requirements in Preparing to deploy OpenShift Data Foundation chapter and follow the appropriate deployment process based on your requirement:

- Deploy OpenShift Data Foundation on Google Cloud
- Deploy standalone Multicloud Object Gateway component

CHAPTER 1. PREPARING TO DEPLOY OPENSHIFT DATA FOUNDATION

Deploying OpenShift Data Foundation on OpenShift Container Platform using dynamic storage devices provides you with the option to create internal cluster resources. This will result in the internal provisioning of the base services, which helps to make additional storage classes available to applications.

Before you begin the deployment of Red Hat OpenShift Data Foundation, follow these steps:

1. Optional: If you want to enable cluster-wide encryption using the external Key Management System (KMS) HashiCorp Vault, follow these steps:
    - Ensure that you have a valid Red Hat OpenShift Data Foundation Advanced subscription. To know how subscriptions for OpenShift Data Foundation work, see knowledgebase article on OpenShift Data Foundation subscriptions.
    - When the Token authentication method is selected for encryption then refer to Enabling cluster-wide encryption with the Token authentication using KMS.
    - When the Kubernetes authentication method is selected for encryption then refer to Enabling cluster-wide encryption with the Kubernetes authentication using KMS .
    - Ensure that you are using signed certificates on your Vault servers.
2. Optional: If you want to enable cluster-wide encryption using the external Key Management System (KMS) Thales CipherTrust Manager, you must first enable the Key Management Interoperability Protocol (KMIP) and use signed certificates on your server. Follow these steps:
    a. Create a KMIP client if one does not exist. From the user interface, select **KMIP → Client Profile → Add Profile**.
        i. Add the **CipherTrust** username to the **Common Name** field during profile creation.
    b. Create a token by navigating to **KMIP → Registration Token → New Registration Token**. Copy the token for the next step.
    c. To register the client, navigate to **KMIP → Registered Clients → Add Client**. Specify the **Name**. Paste the **Registration Token** from the previous step, then click **Save**.
    d. Download the Private Key and Client Certificate by clicking **Save Private Key** and **Save Certificate** respectively.
    e. To create a new KMIP interface, navigate to **Admin Settings → Interfaces → Add Interface**.
        i. Select **KMIP Key Management Interoperability Protocol** and click **Next**.
        ii. Select a free **Port**.
        iii. Select **Network Interface** as **all**.
        iv. Select **Interface Mode** as **TLS, verify client cert, user name taken from client cert, auth request is optional**.
        v. (Optional) You can enable hard delete to delete both metadata and material when the key is deleted. It is disabled by default.

vi. Select the CA to be used, and click **Save**.

f. To get the server CA certificate, click on the Action menu (⋮) on the right of the newly created interface, and click **Download Certificate**.

g. Optional: If StorageClass encryption is to be enabled during deployment, create a key to act as the Key Encryption Key (KEK):

i. Navigate to **Keys** → **Add Key**.

ii. Enter **Key Name**.

iii. Set the **Algorithm** and **Size** to **AES** and **256** respectively.

iv. Enable **Create a key in Pre-Active state**and set the date and time for activation.

v. Ensure that **Encrypt** and **Decrypt** are enabled under **Key Usage**.

vi. Copy the ID of the newly created Key to be used as the Unique Identifier during deployment.

3. Minimum starting node requirements
An OpenShift Data Foundation cluster will be deployed with minimum configuration when the standard deployment resource requirement is not met. See Resource requirements section in Planning guide.

4. Disaster recovery requirements
Disaster Recovery features supported by Red Hat OpenShift Data Foundation require all of the following prerequisites to successfully implement a disaster recovery solution:

- A valid Red Hat OpenShift Data Foundation Advanced subscription
- A valid Red Hat Advanced Cluster Management for Kubernetes subscription
To know how subscriptions for OpenShift Data Foundation work, see knowledgebase article on OpenShift Data Foundation subscriptions.

For detailed requirements, see Configuring OpenShift Data Foundation Disaster Recovery for OpenShift Workloads guide, and *Requirements and recommendations* section of the Install guide in Red Hat Advanced Cluster Management for Kubernetes documentation.

CHAPTER 2. DEPLOYING OPENSHIFT DATA FOUNDATION ON GOOGLE CLOUD

You can deploy OpenShift Data Foundation on OpenShift Container Platform using dynamic storage devices provided by Google Cloud installer-provisioned infrastructure. This enables you to create internal cluster resources and it results in internal provisioning of the base services, which helps to make additional storage classes available to applications.

Also, it is possible to deploy only the Multicloud Object Gateway (MCG) component with OpenShift Data Foundation. For more information, see Deploy standalone Multicloud Object Gateway .

NOTE

Only internal OpenShift Data Foundation clusters are supported on Google Cloud. See Planning your deployment for more information about deployment requirements.

Ensure that you have addressed the requirements in Preparing to deploy OpenShift Data Foundation chapter before proceeding with the below steps for deploying using dynamic storage devices:

1. Install the Red Hat OpenShift Data Foundation Operator .
2. Create the OpenShift Data Foundation Cluster .

2.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR

You can install Red Hat OpenShift Data Foundation Operator using the Red Hat OpenShift Container Platform Software Catalog.

Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** and operator installation permissions.
- You must have at least three worker or infrastructure nodes in the Red Hat OpenShift Container Platform cluster.
- For additional resource requirements, see the Planning your deployment guide.

IMPORTANT

- When you need to override the cluster-wide default node selector for OpenShift Data Foundation, you can use the following command to specify a blank node selector for the **openshift-storage** namespace (create **openshift-storage** namespace in this case):

```
$ oc annotate namespace openshift-storage openshift.io/node-selector=
```

- Taint a node as **infra** to ensure only Red Hat OpenShift Data Foundation resources are scheduled on that node. This helps you save on subscription costs. For more information, see the *How to use dedicated worker nodes for Red Hat OpenShift Data Foundation* section in the Managing and Allocating Storage Resources guide.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**.
3. Scroll or type **OpenShift Data Foundation** into the **Filter by keyword** box to find the **OpenShift Data Foundation** Operator.
4. Click **Install**.
5. Set the following options on the **Install Operator** page:
   a. Update Channel as **stable-4.21**.
   b. Installation Mode as **A specific namespace on the cluster**.
   c. Installed Namespace as **Operator recommended namespace openshift-storage**. If Namespace **openshift-storage** does not exist, it is created during the operator installation.
   d. Select Approval Strategy as **Automatic** or **Manual**.
      If you select **Automatic** updates, then the Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without any intervention.

      If you select **Manual** updates, then the OLM creates an update request. As a cluster administrator, you must then manually approve that update request to update the Operator to a newer version.
   e. Ensure that the **Enable** option is selected for the **Console plugin**.
   f. Click **Install**.

Verification steps

- After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.
- In the Web Console:
  - Navigate to Installed Operators and verify that the **OpenShift Data Foundation** Operator shows a green tick indicating successful installation.
  - Navigate to **Storage** and verify if the **Data Foundation** dashboard is available.

2.2. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE TOKEN AUTHENTICATION METHOD

You can enable the key value backend path and policy in the vault for token authentication.

Prerequisites

- Administrator access to the vault.
- A valid Red Hat OpenShift Data Foundation Advanced subscription. For more information, see the knowledgebase article on OpenShift Data Foundation subscriptions.

- Carefully, select a unique path name as the backend **path** that follows the naming convention since you cannot change it later.

Procedure

1. Enable the Key/Value (KV) backend path in the vault.
   For vault KV secret engine API, version 1:

   ```
   $ vault secrets enable -path=odf kv
   ```

   For vault KV secret engine API, version 2:

   ```
   $ vault secrets enable -path=odf kv-v2
   ```

2. Create a policy to restrict the users to perform a write or delete operation on the secret:

   ```
   echo '
   path "odf/*" {
     capabilities = ["create", "read", "update", "delete", "list"]
   }
   path "sys/mounts" {
   capabilities = ["read"]
   }'| vault policy write odf -
   ```

3. Create a token that matches the above policy:

   ```
   $ vault token create -policy=odf -format json
   ```

2.3. ENABLING CLUSTER-WIDE ENCRYPTION WITH KMS USING THE KUBERNETES AUTHENTICATION METHOD

You can enable the Kubernetes authentication method for cluster-wide encryption using the Key Management System (KMS).

Prerequisites

- Administrator access to Vault.
- A valid Red Hat OpenShift Data Foundation Advanced subscription. For more information, see the knowledgebase article on OpenShift Data Foundation subscriptions.
- The OpenShift Data Foundation operator must be installed from the Software Catalog.
- Select a unique path name as the backend **path** that follows the naming convention carefully. You cannot change this path name later.

Procedure

1. Create a service account:

   ```
   $ oc -n openshift-storage create serviceaccount <serviceaccount_name>
   ```

where, ***<serviceaccount_name>*** specifies the name of the service account.

For example:

```
$ oc -n openshift-storage create serviceaccount odf-vault-auth
```

2. Create **clusterrolebindings** and **clusterroles**:

```
$ oc -n openshift-storage create clusterrolebinding vault-tokenreview-binding --
clusterrole=system:auth-delegator --serviceaccount=openshift-
storage:_<serviceaccount_name>_
```

For example:

```
$ oc -n openshift-storage create clusterrolebinding vault-tokenreview-binding --
clusterrole=system:auth-delegator --serviceaccount=openshift-storage:odf-vault-auth
```

3. Create a secret for the **serviceaccount** token and CA certificate.

```
$ cat <<EOF | oc create -f -
apiVersion: v1
kind: Secret
metadata:
  name: odf-vault-auth-token
  namespace: openshift-storage
  annotations:
    kubernetes.io/service-account.name: <serviceaccount_name>
type: kubernetes.io/service-account-token
data: {}
EOF
```

where, ***<serviceaccount_name>*** is the service account created in the earlier step.

4. Get the token and the CA certificate from the secret.

```
$ SA_JWT_TOKEN=$(oc -n openshift-storage get secret odf-vault-auth-token -o jsonpath="
{.data['token']}" | base64 --decode; echo)
$ SA_CA_CRT=$(oc -n openshift-storage get secret odf-vault-auth-token -o jsonpath="
{.data['ca\.crt']}" | base64 --decode; echo)
```

5. Retrieve the OCP cluster endpoint.

```
$ OCP_HOST=$(oc config view --minify --flatten -o jsonpath="{.clusters[0].cluster.server}")
```

6. Fetch the service account issuer:

```
$ oc proxy &
$ proxy_pid=$!
$ issuer="$( curl --silent http://127.0.0.1:8001/.well-known/openid-configuration | jq -r
.issuer)"
$ AUDIENCE=$(echo '{"apiVersion": "authentication.k8s.io/v1", "kind": "TokenRequest"}' |
```

```
kubectl create f --raw /api/v1/namespaces/default/serviceaccounts/default/token | jq -r
'.spec.audiences' | jq -r .[0])
$ kill $proxy_pid
```

Or, for Vault version 1.21.0 and higher:

```
$ oc proxy &
$ proxy_pid=$!
$ ISSUER=$(echo '{"apiVersion": "authentication.k8s.io/v1", "kind": "TokenRequest"}' \   |
kubectl create f --raw /api/v1/namespaces/default/serviceaccounts/default/token \   | jq -r
'.status.token' \   | cut -d . -f2 \   | base64 -d \   | jq '.iss')
$ AUDIENCE=$(echo '{"apiVersion": "authentication.k8s.io/v1", "kind": "TokenRequest"}' |
kubectl create f --raw /api/v1/namespaces/default/serviceaccounts/default/token | jq -r
'.spec.audiences' | jq -r .[0])
$ kill $proxy_pid
```

7. Use the information collected in the previous step to setup the Kubernetes authentication method in Vault:

```
$ vault auth enable kubernetes
```

```
$ vault write auth/kubernetes/config \
          token_reviewer_jwt="$SA_JWT_TOKEN" \
          kubernetes_host="$OCP_HOST" \
          kubernetes_ca_cert="$SA_CA_CRT" \
          issuer="$issuer"
```

Or, for Vault version 1.21.0 and higher:

```
$ oc exec -n vault -ti vault-0 – \
    vault write auth/kubernetes/config \
    token_reviewer_jwt="$SA_JWT_TOKEN" \
    kubernetes_host="$OCP_HOST" \
    kubernetes_ca_cert="$SA_CA_CRT" \
    issuer="$ISSUER"\
    token_audience="$AUDIENCE"
```

IMPORTANT

To configure the Kubernetes authentication method in Vault when the issuer is empty:

```
$ vault write auth/kubernetes/config \
          token_reviewer_jwt="$SA_JWT_TOKEN" \
          kubernetes_host="$OCP_HOST" \
          kubernetes_ca_cert="$SA_CA_CRT"
```

8. Enable the Key/Value (KV) backend path in Vault.
For Vault KV secret engine API, version 1:

```
$ vault secrets enable -path=odf kv
```

For Vault KV secret engine API, version 2:

```
$ vault secrets enable -path=odf kv-v2
```

9. Create a policy to restrict the users to perform a **write** or **delete** operation on the secret:

```
echo '
path "odf/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}
path "sys/mounts" {
capabilities = ["read"]
}'| vault policy write odf -
```

10. Generate the roles:

```
$ vault write auth/kubernetes/role/odf-rook-ceph-op \
        bound_service_account_names=rook-ceph-system,rook-ceph-osd,noobaa \
        bound_service_account_namespaces=openshift-storage \
        policies=odf \
        ttl=1440h
```

The role **odf-rook-ceph-op** is later used while you configure the KMS connection details during the creation of the storage system.

```
$ vault write auth/kubernetes/role/odf-rook-ceph-osd \
        bound_service_account_names=rook-ceph-osd \
        bound_service_account_namespaces=openshift-storage \
        policies=odf \
        ttl=1440h
```

2.3.1. Enabling and disabling key rotation when using KMS

Security common practices require periodic encryption of key rotation. You can enable or disable key rotation when using KMS.

2.3.1.1. Enforcing precedence for key rotation

In key rotation, precedence refers to the order in which the system checks for scheduled annotations. In OpenShift Data Foundation, the default precedence is set to storage class (recommended). This means the system reads annotations only from the storage class.

However, if you want the system to check the persistent volume claim (PVC) first and then fall back to the storage class, you can configure this behavior by setting the **schedule-precedence** to PVC in the CSI-addons ConfigMap.

You can define the **ConfigMap** for csi-addons as shown below:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: csi-addons-config
```

```
  namespace: openshift-storage
data:
  "schedule-precedence": "pvc"
```

Restart the **csi-addons** operator pod for the changes to take effect:

```
oc delete po -n openshift-storage -l "app.kubernetes.io/name=csi-addons"
```

2.3.1.2. Enabling key rotation

To enable key rotation, add the annotation **keyrotation.csiaddons.openshift.io/schedule: <value>** to **PersistentVolumeClaims**, **Namespace**, or **StorageClass** (in the decreasing order of precedence).

<value> can be **@hourly**, **@daily**, **@weekly**, **@monthly**, or **@yearly**. If **<value>** is empty, the default is **@weekly**. The below examples use **@weekly**.

IMPORTANT

Key rotation is only supported for RBD backed volumes.

Annotating Namespace

```
$ oc get namespace default
NAME      STATUS   AGE
default   Active   5d2h
```

```
$ oc annotate namespace default "keyrotation.csiaddons.openshift.io/schedule=@weekly"
namespace/default annotated
```

Annotating StorageClass

```
$ oc get storageclass rbd-sc
NAME       PROVISIONER        RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION   AGE
rbd-sc     rbd.csi.ceph.com   Delete          Immediate           true                  5d2h
```

```
$ oc annotate storageclass rbd-sc "keyrotation.csiaddons.openshift.io/schedule=@weekly"
storageclass.storage.k8s.io/rbd-sc annotated
```

Annotating PersistentVolumeClaims

```
$ oc get pvc data-pvc
NAME      STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS     AGE
data-pvc  Bound    pvc-f37b8582-4b04-4676-88dd-e1b95c6abf74   1Gi        RWO           default
20h
```

```
$ oc annotate pvc data-pvc "keyrotation.csiaddons.openshift.io/schedule=@weekly"
persistentvolumeclaim/data-pvc annotated
```

```
$ oc get encryptionkeyrotationcronjobs.csiaddons.openshift.io
NAME                SCHEDULE    SUSPEND   ACTIVE   LASTSCHEDULE   AGE
data-pvc-1642663516   @weekly                                3s
```

```
$ oc annotate pvc data-pvc "keyrotation.csiaddons.openshift.io/schedule=*/1 * * * *" --overwrite=true
persistentvolumeclaim/data-pvc annotated
```

```
$ oc get encryptionkeyrotationcronjobs.csiaddons.openshift.io
NAME               SCHEDULE    SUSPEND   ACTIVE   LASTSCHEDULE   AGE
data-pvc-1642664617   */1 * * * *                            3s
```

2.3.1.3. Disabling key rotation

You can disable key rotation for the following:

- All the persistent volume claims (PVCs) of storage class
- A specific PVC

Disabling key rotation for all PVCs of a storage class

To disable key rotation for all PVCs, update the annotation of the storage class:

```
$ oc get storageclass rbd-sc
NAME      PROVISIONER       RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION   AGE
rbd-sc     rbd.csi.ceph.com   Delete         Immediate          true                  5d2h
```

```
$ oc annotate storageclass rbd-sc "keyrotation.csiaddons.openshift.io/enable: false"
storageclass.storage.k8s.io/rbd-sc annotated
```

Disabling key rotation for a specific persistent volume claim

1. Identify the **EncryptionKeyRotationCronJob** CR for the PVC you want to disable key rotation on:

   ```
   $ oc get encryptionkeyrotationcronjob -o jsonpath='{range .items[?
   (@.spec.jobTemplate.spec.target.persistentVolumeClaim=="<PVC_NAME>")]}
   {.metadata.name}{"\n"}{end}'
   ```

   Where **<PVC_NAME>** is the name of the PVC that you want to disable.

2. Apply the following to the **EncryptionKeyRotationCronJob** CR from the previous step to disable the key rotation:

   a. Update the **csiaddons.openshift.io/state** annotation from **managed** to **unmanaged**:

      ```
      $ oc annotate encryptionkeyrotationcronjob <encryptionkeyrotationcronjob_name>
      "csiaddons.openshift.io/state=unmanaged" --overwrite=true
      ```

      Where <encryptionkeyrotationcronjob_name> is the name of the **EncryptionKeyRotationCronJob** CR.

b. Add **suspend: true** under the **spec** field:

```
$ oc patch encryptionkeyrotationcronjob <encryptionkeyrotationcronjob_name> -p
'{"spec": {"suspend": true}}' --type=merge.
```

3. Save and exit. The key rotation will be disabled for the PVC.

2.4. CREATING OPENSHIFT DATA FOUNDATION CLUSTER

Create an OpenShift Data Foundation cluster after you install the OpenShift Data Foundation operator.

Prerequisites

- The OpenShift Data Foundation operator must be installed from the Software Catalog. For more information, see Installing OpenShift Data Foundation Operator.

- Be aware that the default storage class of the Google Cloud platform uses hard disk drive (HDD). To use solid state drive (SSD) based disks for better performance, you need to create a storage class, using **pd-ssd** as shown in the following **ssd-storeageclass.yaml** example:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
 name: faster
provisioner: kubernetes.io/gce-pd
parameters:
 type: pd-ssd
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete
```

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation → Storage Systems → Create StorageSystem**.

2. In the **Backing storage** page, select the following:

   a. Select the **Use an existing StorageClass** option.

   b. Select the **Storage Class**.
   By default, it is set as **standard**. However, if you created a storage class to use SSD based disks for better performance, you need to select that storage class.

   c. Click **Next**.

3. In the **Advanced Settings** page, you can configure the following optional items:

   - **Enable Network Files System**
     Automatically enable NFS for the cluster

   - **Use Ceph RBD as the default StorageClass**
     Sets Ceph RBD as the default StorageClass, eliminating the need to manually annotate a StorageClass.

   - **Set default StorageClass for virtualization**

Marks the RBD virtualization storage class as the default for KubeVirt VM disks (PVs) after installation.

- **Use external PostgreSQL** [Technology preview]
  Enables the use of an external PostgreSQL instance. This provides a high-availability solution for the Multicloud Object Gateway, where the PostgreSQL pod is a single point of failure.

  IMPORTANT

  OpenShift Data Foundation ships PostgreSQL images maintained by Red Hat, which are used to store metadata for the Multicloud Object Gateway. This PostgreSQL usage is at the application level.

  As a result, OpenShift Data Foundation does not perform database-level optimizations or in-depth insights.

  If you have well-maintained and optimized PostgreSQL instance, then it is recommended to use it. OpenShift Data Foundation supports external PostgreSQL instances.

  Any PostgreSQL-related issues requiring code changes or deep technical analysis may need to be addressed upstream. This could result in longer resolution times.

  a. Provide the following connection details:
     - **Username**
     - **Password**
     - **Server name** and **Port**
     - **Database name**
  b. Select **Enable TLS/SSL** checkbox to enable encryption for the Postgres server.

- **Enable automatic backup**
  Allows automatic backup for the Multicloud Object Gateway metadata database.

  a. Select the backup frequency: Daily, Weekly, or Monthly.
  b. Specify the number of backups to retain.

     NOTE

     This option is disabled if the **Use external PostgreSQL** option is selected.

  c. Click **Next**.

4. In the **Capacity and nodes** page, provide the necessary information:

   a. Select a value for **Requested Capacity** from the dropdown list. It is set to **2 TiB** by default.

> **NOTE**
>
> Once you select the initial storage capacity, cluster expansion is performed only using the selected usable capacity (three times of raw storage).

b. In the **Select Nodes** section, select at least three available nodes.

c. In the **Configure performance** section, select one of the following performance profiles:

- Lean
  Use this in a resource constrained environment with minimum resources that are lower than the recommended. This profile minimizes resource consumption by allocating fewer CPUs and less memory.

- Balanced (default)
  Use this when recommended resources are available. This profile provides a balance between resource consumption and performance for diverse workloads.

- Performance
  Use this in an environment with sufficient resources to get the best performance. This profile is tailored for high performance by allocating ample memory and CPUs to ensure optimal execution of demanding workloads.

> **NOTE**
>
> You have the option to configure the performance profile even after the deployment using the **Configure performance** option from the options menu of the **StorageSystems** tab.

> **IMPORTANT**
>
> Before selecting a resource profile, make sure to check the current availability of resources within the cluster. Opting for a higher resource profile in a cluster with insufficient resources might lead to installation failures.

For more information about resource requirements, see Resource requirement for performance profiles.

d. Optional: Select the **Taint nodes** checkbox to dedicate the selected nodes for OpenShift Data Foundation.
For cloud platforms with multiple availability zones, ensure that the Nodes are spread across different Locations/availability zones.

If the nodes selected do not match the OpenShift Data Foundation cluster requirements of an aggregated 30 CPUs and 72 GiB of RAM, a minimal cluster is deployed. For minimum starting node requirements, see the Resource requirements section in the *Planning* guide.

e. Optional: Select the **Enable automatic capacity scaling for your cluster**checkbox.
When automatic capacity scaling is enabled, additional raw capacity equivalent to the configured deployment size is automatically added to the cluster when used capacity reaches 70%. This ensures your deployment scales seamlessly to meet demand.

This option is disabled in lean profile mode, LSO deployment, and external mode deployment.

IMPORTANT

This may incur additional costs for the underlying storage.

i. Set the cluster expansion limit from the dropdown. This is the maximum the cluster can expand in the cloud. Automatic scaling is suspended if this limit is exceeded.

f. Click **Next**.

5. Optional: In the **Security and network** page, configure the following based on your requirements:

a. To enable encryption, select **Enable data encryption for block and file storage**

i. Select either one or both the encryption levels:

- **Cluster-wide encryption**
  Encrypts the entire cluster (block and file).

- **StorageClass encryption**
  Creates encrypted persistent volume (block only) using encryption enabled storage class.

ii. Optional: Select the **Connect to an external key management service** checkbox. This is optional for cluster-wide encryption.

A. From the **Key Management Service Provider** drop-down list, select one of the following providers and provide the necessary details:

- **Vault**

  I. Select an **Authentication Method**.

  - **Using Token authentication method**
    - Enter a unique **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Token**.
    - Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
      - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
    - Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace**.
    - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key**.
    - Click **Save**.
  - **Using Kubernetes authentication method**
    - Enter a unique Vault **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Role** name.

- Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
    - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
    - Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace** if applicable.
    - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .
- Click **Save**.

> **NOTE**
>
> In case you need to enable key rotation for Vault KMS, run the following command in the OpenShift web console after the storage cluster is created:
>
> ```
> $ oc patch storagecluster ocs-storagecluster -n
> openshift-storage --type=json -p '[{"op": "add",
> "path":"/spec/encryption/keyRotation/enable",
> "value": true}]'
> ```

- **Thales CipherTrust Manager (using KMIP)**
    I. Enter a unique **Connection Name** for the Key Management service within the project.
    II. In the **Address** and **Port** sections, enter the IP of Thales CipherTrust Manager and the port where the KMIP interface is enabled. For example:
        - **Address**: 123.34.3.2
        - **Port**: 5696
    III. Upload the **Client Certificate**, **CA certificate**, and **Client Private Key**.
    IV. If StorageClass encryption is enabled, enter the Unique Identifier to be used for encryption and decryption generated above.
    V. The **TLS Server** field is optional and used when there is no DNS entry for the KMIP endpoint. For example, **kmip_all_<port>.ciphertrustmanager.local**.

b. To enable in-transit encryption, select **In-transit encryption**.
    i. Select a **Network**.
    ii. Click **Next**.

6. In the **Review and create** page, review the configuration details.
   To modify any configuration settings, click **Back**.
7. Click **Create StorageSystem**.

NOTE

When your deployment has five or more nodes, racks, or rooms, and when there are five or more number of failure domains present in the deployment, you can configure Ceph monitor counts based on the number of racks or zones. An alert is displayed in the notification panel or Alert Center of the OpenShift Web Console to indicate the option to increase the number of Ceph monitor counts. You can use the **Configure** option in the alert to configure the Ceph monitor counts. For more information, see Resolving low Ceph monitor count alert.

Verification steps

- To verify the final Status of the installed storage cluster:
    a. In the OpenShift Web Console, navigate to **Storage → Data Foundation → Storage System → ocs-storagecluster**.
    b. Verify that **Status** of **StorageCluster** is **Ready** and has a green tick mark next to it.
- To verify that all components for OpenShift Data Foundation are successfully installed, see Verifying your OpenShift Data Foundation deployment .

Additional resources

To enable Overprovision Control alerts, refer to Alerts in Monitoring guide.
ta_foundation/index#alerts[Alerts] in Monitoring guide.

2.5. VERIFYING OPENSHIFT DATA FOUNDATION DEPLOYMENT

Use this section to verify that OpenShift Data Foundation is deployed correctly.

2.5.1. Verifying the state of the pods

Procedure

1. Click **Workloads → Pods** from the OpenShift Web Console.
2. Select **openshift-storage** from the **Project** drop-down list.

    NOTE

    If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

    For more information on the expected number of pods for each component and how it varies depending on the number of nodes, see the following table:

1. Set filter for Running and Completed pods to verify that the following pods are in **Running** and **Completed** state:

NOTE

The available pods depend on the cluster configuration. When the cluster is deployed as a standalone Multicloud Object Gateway, the **rook-ceph-operator-*** pods are not available. Similarly, when the cluster is deployed without the Multicloud Object Gateway, **noobaa-*** pods are not available.

Component	Corresponding pods
OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any storage node) • **ocs-metrics-exporter-*** (1 pod on any storage node) • **odf-operator-controller-manager-*** (1 pod on any storage node) • **odf-console-*** (1 pod on any storage node) • **csi-addons-controller-manager-*** (1 pod on any storage node) • **ux-backend-server-*** (1 pod on any storage node) • * **ocs-client-operator**-* (1 pod on any storage node) • **ocs-client-operator-console**-* (1 pod on any storage node) • **ocs-provider-server**-* (1 pod on any storage node)
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any storage node)
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any storage node) • **noobaa-core-*** (1 pod on any storage node) • **noobaa-db-pg-cluster-1** and noobaa-db-pg-cluster-2 (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node)

MON	**rook-ceph-mon-*** (3 pods distributed across storage nodes)
MGR	**rook-ceph-mgr-*** (2 pods on different storage nodes, one active, one standby)
MDS	**rook-ceph-mds-ocs-storagecluster-cephfilesystem-*** (2 pods distributed across storage nodes)
CSI	• **cephfs** ◦ **openshift-storage.cephfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.cephfs.csi.ceph.com-nodeplugin-*** (1 pod on any storage nodes) • **nfs** ◦ **openshift-storage.nfs.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.nfs.csi.ceph.com-nodeplugin-*** (1 pod on any storage node) • **rbd** ◦ **openshift-storage.rbd.csi.ceph.com-ctrlplugin-*** (2 pods distributed across storage nodes) ◦ **openshift-storage.rbd.csi.ceph.com-nodeplugin-*** (1 pod on any storage node)
rook-ceph-crashcollector	**rook-ceph-crashcollector-*** (1 pod on each storage node)

OSD	• **rook-ceph-osd-*** (1 pod for each device) • **rook-ceph-osd-prepare-ocs-*** (1 pod for each device)
ceph-csi-operator	**ceph-csi-controller-manager-*** (1 pod for each device)
odf-blackbox-exporter	**odf-blackbox-exporter-*** (1 pod on any storage node) **NOTE:** This does not apply to clusters without storage, for example, Multicloud Object Gateway-only deployments or external mode deployments.

2.5.2. Verifying the OpenShift Data Foundation cluster is healthy

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.
3. In the **Status** card of the **Block and File** tab, verify that the *Storage Cluster* has a green tick.
4. In the **Details** card, verify that the cluster information is displayed.

For more information on the health of the OpenShift Data Foundation cluster using the **Block and File** dashboard, see Monitoring OpenShift Data Foundation.

2.5.3. Verifying the Multicloud Object Gateway is healthy

Procedure

1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.
    a. In the **Status card** of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.
    b. In the **Details** card, verify that the MCG information is displayed.

For more information on the health of the OpenShift Data Foundation cluster using the object service dashboard, see Monitoring OpenShift Data Foundation.

IMPORTANT

To avoid data loss, it is recommended to take a backup of NooBaa DB PVC regularly. If NooBaa DB fails and cannot be recovered, then you can revert to the latest backed-up version. For instructions on backing up your NooBaa DB, follow the steps in the knowledgebase article, Perform a One-Time Backup of the Database for the Multicloud Object Gateway.

2.5.4. Verifying that the specific storage classes exist

Procedure

1. Click **Storage → Storage Classes** from the left pane of the OpenShift Web Console.
2. Verify that the following storage classes are created with the OpenShift Data Foundation cluster creation:
   - **ocs-storagecluster-ceph-rbd**
   - **ocs-storagecluster-cephfs**
   - **openshift-storage.noobaa.io**

CHAPTER 3. DEPLOY STANDALONE MULTICLOUD OBJECT GATEWAY

Deploying only the Multicloud Object Gateway component with the OpenShift Data Foundation provides the flexibility in deployment and helps to reduce the resource consumption. After deploying the MCG component, you can create and manage buckets using MCG object browser. For more information, see Creating and managing buckets using MCG object browser .

Use this section to deploy only the standalone Multicloud Object Gateway component, which involves the following steps:

- Installing Red Hat OpenShift Data Foundation Operator
- Creating standalone Multicloud Object Gateway

> **IMPORTANT**
>
> To avoid data loss, it is recommended to take a backup of NooBaa DB PVC regularly. If NooBaa DB fails and cannot be recovered, then you can revert to the latest backed-up version. For instructions on backing up NooBaa DB, follow the steps in the knowledgebase article, Backup and Restore for Multicloud Object Gateway database (NooBaa DB).

3.1. INSTALLING RED HAT OPENSHIFT DATA FOUNDATION OPERATOR

You can install Red Hat OpenShift Data Foundation Operator using the Red Hat OpenShift Container Platform Software Catalog.

Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** and operator installation permissions.
- You must have at least three worker or infrastructure nodes in the Red Hat OpenShift Container Platform cluster.
- For additional resource requirements, see the Planning your deployment guide.

> **IMPORTANT**
>
> - When you need to override the cluster-wide default node selector for OpenShift Data Foundation, you can use the following command to specify a blank node selector for the **openshift-storage** namespace (create **openshift-storage** namespace in this case):
>
> ```
> $ oc annotate namespace openshift-storage openshift.io/node-selector=
> ```
>
> - Taint a node as **infra** to ensure only Red Hat OpenShift Data Foundation resources are scheduled on that node. This helps you save on subscription costs. For more information, see the *How to use dedicated worker nodes for Red Hat OpenShift Data Foundation* section in the Managing and Allocating Storage Resources guide.

Procedure

1. Log in to the OpenShift Web Console.
2. Click **Ecosystem → Software Catalog**
3. Scroll or type **OpenShift Data Foundation** into the **Filter by keyword** box to find the **OpenShift Data Foundation** Operator.
4. Click **Install**.
5. Set the following options on the **Install Operator** page:
    a. Update Channel as **stable-4.21**.
    b. Installation Mode as **A specific namespace on the cluster**
    c. Installed Namespace as **Operator recommended namespace openshift-storage**. If Namespace **openshift-storage** does not exist, it is created during the operator installation.
    d. Select Approval Strategy as **Automatic** or **Manual**.
    If you select **Automatic** updates, then the Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without any intervention.

    If you select **Manual** updates, then the OLM creates an update request. As a cluster administrator, you must then manually approve that update request to update the Operator to a newer version.
    e. Ensure that the **Enable** option is selected for the **Console plugin**.
    f. Click **Install**.

Verification steps

- After the operator is successfully installed, the web console automatically reloads to apply the changes. During this process, a temporary error message might appear on the page and this is expected and disappears after the refresh completes.
- In the Web Console:
    - Navigate to Installed Operators and verify that the **OpenShift Data Foundation** Operator shows a green tick indicating successful installation.
    - Navigate to **Storage** and verify if the **Data Foundation** dashboard is available.

3.2. CREATING A STANDALONE MULTICLOUD OBJECT GATEWAY

You can create only the standalone Multicloud Object Gateway (MCG) component while deploying OpenShift Data Foundation. After you create the MCG component, you can create and manage buckets using the MCG object browser. For more information, see Creating and managing buckets using MCG object browser.

Prerequisites

- Ensure that the OpenShift Data Foundation Operator is installed.

- The Multicloud Object Gateway database uses ReadWriteOncePod (RWOP) access mode to safeguard against node failures. If the underlying CSI driver does not support RWOP, the access mode must be manually changed to ReadWriteOnce (RWO) before deployment. For more information, see the knowledgebase article How to change NooBaa DB access mode from RWOP to RWO in standalone MCG deployments?.

Procedure

1. In the OpenShift Web Console, navigate to **Storage → StorageCluster**.
2. On the Welcome page, click Configure Data Foundation.
3. Select Setup Multicloud Object Gateway.
4. In the **Backing storage** page, select the following:
   a. Select the **Use an existing StorageClass** option.
   b. Click **Next**.
5. In the **Advanced Settings** page, you can configure the following optional items:
   - **Use external PostgreSQL** [Technology preview]
     Enables the use of an external PostgreSQL instance. This provides a high-availability solution for the Multicloud Object Gateway, where the PostgreSQL pod is a single point of failure.

     IMPORTANT

     OpenShift Data Foundation ships PostgreSQL images maintained by Red Hat, which are used to store metadata for the Multicloud Object Gateway. This PostgreSQL usage is at the application level.

     As a result, OpenShift Data Foundation does not perform database-level optimizations or in-depth insights.

     If you have well-maintained and optimized PostgreSQL instance, then it is recommended to use it. OpenShift Data Foundation supports external PostgreSQL instances.

     Any PostgreSQL-related issues requiring code changes or deep technical analysis may need to be addressed upstream. This could result in longer resolution times.

     a. Provide the following connection details:
        - **Username**
        - **Password**
        - **Server name** and **Port**
        - **Database name**

     b. Select **Enable TLS/SSL** checkbox to enable encryption for the Postgres server.
   - **Enable automatic backup**

Allows automatic backup for the NooBaa database.

a. Select the backup frequency: Daily, Weekly, or Monthly.

b. Specify the number of backups to retain.

c. Select a VolumeSnapshotClass to add a volume snapshot.

> **NOTE**
>
> This option is disabled if the **Use external PostgreSQL** option is selected.

d. Click **Next**.

6. In the **Security** page, select any of the following optional items:

   a. Select the **Connect to an external key management service** checkbox. This is optional for cluster-wide encryption.

   b. From the **Key Management Service Provider** drop-down list, either select **Vault** or **Thales CipherTrust Manager (using KMIP)**. If you selected **Vault**, go to the next step. If you selected **Thales CipherTrust Manager (using KMIP)**, go to step iii.

   c. Select an **Authentication Method**.

      Using Token authentication method

      - Enter a unique **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Token**.
      - Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
        - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.
        - Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace**.
        - Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .
        - Click **Save** and skip to step iv.

      Using Kubernetes authentication method

      - Enter a unique Vault **Connection Name**, host **Address** of the Vault server ('https://<hostname or ip>'), **Port** number and **Role** name.
      - Expand **Advanced Settings** to enter additional settings and certificate details based on your **Vault** configuration:
        - Enter the Key Value secret path in **Backend Path** that is dedicated and unique to OpenShift Data Foundation.

- Optional: Enter **TLS Server Name**, **Authentication Path**, and **Vault Enterprise Namespace** if applicable.
- Upload the respective PEM encoded certificate file to provide the **CA Certificate**, **Client Certificate** and **Client Private Key** .
- Click **Save** and skip to step iv.

d. To use **Thales CipherTrust Manager (using KMIP)** as the KMS provider, follow the steps below:

i. Enter a unique **Connection Name** for the Key Management service within the project.

ii. In the **Address** and **Port** sections, enter the IP of Thales CipherTrust Manager and the port where the KMIP interface is enabled. For example:

- **Address**: 123.34.3.2
- **Port**: 5696

iii. Upload the **Client Certificate**, **CA certificate**, and **Client Private Key**.

iv. If StorageClass encryption is enabled, enter the Unique Identifier to be used for encryption and decryption generated above.

v. The **TLS Server** field is optional and used when there is no DNS entry for the KMIP endpoint. For example, **kmip_all_<port>.ciphertrustmanager.local**.

e. Select a **Network**.

f. Click **Next**.

7. In the **Review and create** page, review the configuration details:
To modify any configuration settings, click **Back**.

8. Click **Create StorageSystem**.

Verification steps

Verifying that the OpenShift Data Foundation cluster is healthy

1. In the OpenShift Web Console, click **Storage → Data Foundation**.
2. In the **Status** card of the **Overview** tab, click **Storage System** and then click the storage system link from the pop up that appears.

   a. In the **Status card** of the **Object** tab, verify that both *Object Service* and *Data Resiliency* have a green tick.

   b. In the **Details** card, verify that the MCG information is displayed.

Verifying the state of the pods

1. Click **Workloads → Pods** from the OpenShift Web Console.

2. Select **openshift-storage** from the **Project** drop-down list and verify that the following pods are in **Running** state.

> **NOTE**
>
> If the **Show default projects** option is disabled, use the toggle button to list all the default projects.

Component	Corresponding pods
OpenShift Data Foundation Operator	• **ocs-operator-*** (1 pod on any storage node) • **ocs-metrics-exporter-*** (1 pod on any storage node) • **odf-operator-controller-manager-*** (1 pod on any storage node) • **odf-console-*** (1 pod on any storage node) • **csi-addons-controller-manager-*** (1 pod on any storage node)
Rook-ceph Operator	**rook-ceph-operator-*** (1 pod on any storage node)
Multicloud Object Gateway	• **noobaa-operator-*** (1 pod on any storage node) • **noobaa-core-*** (1 pod on any storage node) • **noobaa-db-pg-cluster-1** and noobaa-db-pg-cluster-2 (2 instances of MCG DB pod on any storage node) • **noobaa-endpoint-*** (1 pod on any storage node) • **cnpg-controller-manager-*** (1 pod on any storage node)

CHAPTER 4. VIEW OPENSHIFT DATA FOUNDATION TOPOLOGY

The topology shows the mapped visualization of the OpenShift Data Foundation storage cluster at various abstraction levels and also lets you to interact with these layers. The view also shows how the various elements compose the Storage cluster altogether.

Procedure

1. On the OpenShift Web Console, navigate to **Storage → Data Foundation → Topology**.
   The view shows the storage cluster and the zones inside it. You can see the nodes depicted by circular entities within the zones, which are indicated by dotted lines. The label of each item or resource contains basic information such as status and health or indication for alerts.

2. Choose a node to view node details on the right-hand panel. You can also access resources or deployments within a node by clicking on the search/preview decorator icon.

3. To view deployment details

   a. Click the preview decorator on a node. A modal window appears above the node that displays all of the deployments associated with that node along with their statuses.

   b. Click the **Back to main view**button in the model's upper left corner to close and return to the previous view.

   c. Select a specific deployment to see more information about it. All relevant data is shown in the side panel.

4. Click the **Resources** tab to view the pods information. This tab provides a deeper understanding of the problems and offers granularity that aids in better troubleshooting.

5. Click the pod links to view the pod information page on OpenShift Container Platform. The link opens in a new window.

CHAPTER 5. UNINSTALLING OPENSHIFT DATA FOUNDATION

5.1. UNINSTALLING OPENSHIFT DATA FOUNDATION IN INTERNAL MODE

To uninstall OpenShift Data Foundation in Internal mode, refer to the knowledgebase article on Uninstalling OpenShift Data Foundation.